Agentic AI
Lecture Notes
Lab: Build a RAG System – from Retrieval to Agentic RAG

Dr. Jawwad Ahmed Shamsi
School of Computing, FAST–NUCES, Karachi

November 4, 2026

Abstract

In this lab you will build a complete Retrieval-Augmented Generation system step by step, following the RAG lecture notes: chunking, embedding and retrieval, measuring retrieval quality, generating grounded answers with citations, hybrid search with reciprocal rank fusion, HyDE, and a small "lost in the middle" experiment. The final challenge turns retrieval into a *tool* that the model decides when to use – agentic RAG – with a Self-RAG-style check that every sentence of the answer is supported. Everything runs on Google Colab with a free GPU; no API key is needed. The tasks are in increasing order of difficulty; complete as many as you can in class and finish the rest at home.

Before You Start

- Open the notebook `rag_lab_student.ipynb` in Google Colab. You may work in pairs.
- Select **Runtime** → **Change runtime type** → **T4 GPU**, then run the Task 0 cells. The first run downloads two models (about 3 GB) and takes a few minutes.
- The models are `all-MiniLM-L6-v2` (embeddings) and `Qwen2.5-1.5B-Instruct` (a small LLM for generation). The document is a short *sample university handbook* (attendance, exams, grading, fee refunds, library, academic integrity, transcripts, hostel) written for this lab, so the correct answers are known.
- Every cell marked `# TODO` has a line `... # <- your code here` that you replace with your own code. Run the cells **in order**: later tasks use functions from earlier ones.
- Record your results and answers to the *Record in Your Report* questions as you go.

Caveat

A 1.5-billion-parameter model is small. It will sometimes ignore instructions, give a slightly wrong answer, or produce a different result from your neighbour's. That is part of the lab: you are observing how retrieval quality and prompt design change the behaviour of a real (small) model.

Task 0: Setup

Run the setup cells. The last one prints a short greeting from the LLM – if you see it, the models are loaded. Look at the two helper functions you will use throughout:

Jawwad A Shamsi

- `embed(texts)` returns one normalised vector per text, so a dot product equals cosine similarity;
- `generate(messages)` sends a list of chat messages to the LLM and returns its reply.

Task 1: Chunking

Complete `chunk_words(text, size, overlap)` so that it splits the handbook into pieces of `size` words, starting a new piece every `size - overlap` words. Print the first three chunks.

Check Your Understanding

Record in your report:

(a) How many chunks do you get with `size=100, overlap=20`?

(b) Find a chunk that begins or ends in the middle of a sentence. What information might be lost there, and how does the overlap help?

Task 2: Embed and Retrieve

Complete `retrieve(question, k)`: compute the similarity of the question's vector with every chunk vector and return the indices of the `k` best chunks. Try several questions of your own.

Check Your Understanding

Record in your report:

(a) For "How many books can I borrow from the library?", which chunk is ranked first, and what is its score?

(b) Ask a question using words that do *not* appear in the handbook (e.g. "Can I bring my laptop charger into the exam?"). What is retrieved, and why?

Task 3: Evaluate Retrieval with Recall@k

The notebook contains six test questions. Each comes with an *evidence* string that appears only in the passage holding the answer. Complete `recall_at_k` so that a question counts as a hit when any of the top-k chunks contains its evidence. Then run the chunk-size comparison.

Check Your Understanding

Record in your report: fill in this table, then answer the questions.

Chunk size (words)	Number of chunks	Recall@1	Recall@3
40			
100			
250			

Jawwad A. Shamsi

(a) Which chunk size gives the best Recall@3? Is the largest size automatically the best choice for answering questions? (Think about how much text you then put in the prompt.)

(b) Which of the six questions fail most often? Look at the retrieved chunks and suggest why.

Task 4: Generate a Grounded Answer with Citations

Complete `build_prompt(question, passages)` so that the passages are numbered `[1]`, `[2]`, ... followed by the question. The system message (already written) tells the model to answer only from the context, to cite passages, and to say "I could not find this in the documents" if the answer is not there.

Run `rag_answer` on "Can I take a reference book home?" and on the question whose answer is *not* in the handbook ("What is the hostel fee for international students?").

Check Your Understanding

Record in your report:

(a) Did the model cite the correct passage? Check by reading the passage yourself.

(b) For the hostel-fee question, did the model correctly say it could not find the answer, or did it invent one?

(c) Remove the sentence "If the answer is not in the context, say exactly..." from the system message and ask the hostel-fee question again. What changes? What does this tell you about prompt design in RAG?

Task 5: Hybrid Search with Reciprocal Rank Fusion

BM25 keyword search is already set up. Complete `rrf(ranked_lists)`: for every list and every position (rank starting at 1), add $1/(60 + \text{rank})$ to that chunk's score – exactly as in the worked example in the notes. `hybrid_retrieve` then merges the dense and BM25 lists. Run the comparison for the query "F-4015".

Check Your Understanding

Record in your report:

(a) Which methods (dense, BM25, hybrid) place the chunk containing "F-4015" in their top 3?

(b) Compare Recall@3 for dense search (Task 3) and for hybrid search. Did hybrid search help on the six test questions? Why might the gain be small on this short document but large on a real document collection?

Jawwad Shamsi

Task 6: HyDE

Complete `hyde_retrieve`: ask the LLM to write a short handbook-style passage answering the question, embed that passage, and search with its embedding instead of the question's. Compare the rank of the correct passage for the vague question "Can I get my money back if I drop out?" with plain search and with HyDE.

Check Your Understanding

Record in your report:

(a) Copy the hypothetical passage the LLM wrote. Does it contain facts that are wrong according to the handbook?

(b) At which rank did the correct passage appear with plain search, and with HyDE? Explain the difference – or explain why there was none.

Task 7: Lost in the Middle

Here every sentence of the handbook is a separate passage (about 40 passages). For each of three questions, the one passage containing the answer is placed first, in the middle, or last among the others. Complete the line that builds the list of passages, then run the experiment.

Check Your Understanding

Record in your report:

(a) Copy the table of results (first / middle / last) for the three questions.

(b) Do you see the pattern reported in the "Lost in the Middle" paper? With only three questions and a small model, how confident can you be? What would you change to make this experiment more reliable?

Task 8 (Challenge): Agentic RAG with a Support Check

So far the harness always retrieved before answering. Now the **model decides**. It must reply with exactly one line: either `SEARCH: <query>` or `ANSWER: <answer with citations>`. Your harness runs each search with `hybrid_retrieve` and shows the results to the model, numbered so that the numbering continues across searches. This is the ReAct loop from earlier in the course, with search as the only action.

Complete the marked part of `agent()`, then run it on the two-part question in the notebook. Finally, `check_support()` asks the model, sentence by sentence, whether the answer is supported by the passages it saw – a simple, training-free version of Self-RAG's `[IsSup]` check.

Jawwad A Shamsi

Check Your Understanding

Record in your report:

(a) Copy the agent's steps. How many searches did it make, and were its search queries sensible?

(b) Did any sentence fail the support check? If so, was the sentence really unsupported, or was the checker wrong?

(c) Ask the agent a question that needs *no* handbook information (e.g. "What is 2+2?"). Does it still search? Is that a problem?

(d) The agent's reply format (`SEARCH:`/`ANSWER:`) is a home-made protocol. In the next lecture we replace it with *function calling*. What problems with the home-made format did you notice that a standard format would solve?

Try It Yourself

Upload a PDF of your own (e.g. a course handbook or a set of notes), extract its text with `pypdf`, re-run Tasks 1–5 on it, and write five test questions with evidence strings. Report Recall@3 for dense and hybrid search.

Troubleshooting

Table 1: Common problems in this lab.

Symptom	Fix
`CUDA` / out-of-memory error, or generation is very slow	Check that the runtime is T4 GPU; restart the runtime and run Task 0 again.
`NameError` (e.g. `scores` is not defined)	A `TODO` in this or an earlier cell is not completed, or cells were run out of order.
`TypeError: 'NoneType'...` from `build_prompt`	Your function does not `return` the prompt.
Recall is 0 for every question	Check that `retrieve` returns the *highest* scores (sort in descending order).
The agent never writes `SEARCH:` or `ANSWER:`	Small models drift; re-run, or make the system message stricter. Note what happened in your report.

What to Submit

- Your completed notebook (**File** → **Download** → **.ipynb**) with all outputs visible.
- A short report (2–3 pages) containing the Task 3 table and your answers to the *Record in your report* questions for every task you completed.
- (Challenge) The output of your agent for Task 8, and your answers to its questions.

Jawwad A. Shamsi